02

Scale From Zero To Millions Of Users

Keeping a site online is challenging. If you have only one server, everything is simple, but it can't scale. Vertical scaling is straightforward when traffic is low, but it has limits. Horizontal scaling requires load balancers and distributed data, but it lets you add capacity as needed. This chapter traces a single server through caching, CDNs, queues, and sharding.

Single server setup

Everything on one machine: web server, database, cache, and files. This is fine for development and for small traffic. Understand what each component does and what breaks when it fails.

Figure 1

A request flows from a browser or mobile app through DNS to your IP address, then to the web server. The web server reads and writes to the database and serves static files. Dynamic content may be cached in memory to avoid repeated database queries.

- Users connect to your IP address, typically through HTTPS (port 443) for encrypted traffic.
- DNS maps your domain name to your IP. Changes can take minutes to hours to propagate.
- The web server handles HTTP requests and returns HTML, JSON, or files.
- Business logic runs here: authentication, API endpoints, and page rendering.
- The database stores persistent data. Start with SQL unless you have a specific reason for something else.
- Cache frequently accessed data in memory to avoid repeated database queries.
- Static files (images, CSS, JS) can be served from the same server initially.

Database

Most apps need more than one table. Relational databases (SQL) store data in tables with rows and columns and enforce relationships between them. Schemas define the structure, and indexes speed up lookups.

Figure 2

Which database to use?

Most production systems use a relational database as a default. Alternatives matter for specific workloads, not because SQL is bad.

- **Relational (SQL):** Tables with rows and columns, strong consistency, and flexible queries. Handles most workloads including transactions, reporting, and ad-hoc queries. Examples: PostgreSQL, MySQL, SQLite.
- **Document:** Stores nested objects as single documents. Useful when data is read as a whole unit and rarely joined across records. Examples: MongoDB, DynamoDB.
- **Key-value:** Simple lookups by key. Fast and scalable for caching, sessions, and feature flags. Examples: Redis, Memcached.
- **Time-series:** Optimized for metrics and logs ordered by time. Examples: InfluxDB, TimescaleDB.
- **Search:** Full-text search, ranking, and fuzzy matching. Examples: Elasticsearch, Meilisearch.

Start with SQL. Add specialized stores when profiling shows a specific bottleneck.

Schema design basics

- Normalize to avoid duplicate data that can get out of sync.
- Denormalize selectively when read performance matters more than write consistency.
- Add indexes on columns you filter and sort on. Each index slows writes and uses storage.
- Use migrations to change schemas safely in production.

Vertical scaling vs horizontal scaling

Vertical scaling means a bigger machine. Horizontal scaling means more machines. Vertical scaling is simple but has a ceiling. Horizontal scaling removes the ceiling but adds complexity: load balancers, data consistency, and network failures.

- Vertical scaling: more CPU, RAM, or disk on one machine.
- Horizontal scaling: more servers behind a load balancer.
- Stateless servers make horizontal scaling easy. Sessions, uploads, and cache must live outside the web server.
- Databases scale differently. Read replicas help with read-heavy workloads; sharding splits write traffic across multiple primaries.

Load balancer

A load balancer sends traffic to many servers so no single one gets overwhelmed. Clients connect to the balancer, which forwards requests to healthy servers in the pool.

Figure 3

A client's request reaches the load balancer, which picks a healthy server from the pool and forwards the request. The chosen server processes the request and sends the response back through the load balancer. Clients never talk to individual servers directly.

- Health checks remove unhealthy servers automatically. Failed servers stay out of rotation until they pass checks again.
- Round robin distributes requests evenly across servers. Simple and effective when servers have similar capacity.
- Sticky sessions route a user to the same server for multiple requests. Avoid this if possible, it breaks even distribution.
- Terminate TLS at the load balancer to offload encryption from your servers.
- Put the load balancer in front of both web servers and API servers.

Database replication

Read replicas keep copies of the primary database. Most traffic reads from replicas, leaving the primary free for writes. Replication lag means replicas may return slightly stale data.

Figure 4

Writes go to the primary and then replicate to three read replicas. The web server reads from the replicas to spread load evenly.

- Replication lag is the delay between a write on the primary and its appearance on a replica. Usually milliseconds, but can spike under load.
- Read your own writes by routing a user's recent writes to the primary for a short window, or by tracking a version number.
- Failover promotes a replica to primary when the primary dies. Automated failover is faster but riskier than manual failover.
- Connection pooling reuses database connections instead of opening new ones per request.

Cache

Cache stores the results of expensive operations so you don't repeat them on every request. Most web apps read far more than they write, so even a short cache can cut database load dramatically.

Figure 5

The web server checks the cache first. On a hit, it returns the stored result. On a miss, it queries the database, stores the result in the cache, and returns it. Subsequent requests for the same data will hit the cache until the entry expires.

- Cache-aside: the application reads from cache and writes misses itself. Most common pattern.
- Write-through: writes go to cache and database together. Slower writes but cache stays fresh.
- Write-behind: writes go to cache first and sync later. Faster writes but risk losing data if the cache crashes.
- Set TTLs to bound how long stale data lives. Pick shorter TTLs for data that changes frequently.
- Invalidate cache entries when the underlying data changes, or users will see stale results.
- Redis and Memcached are popular in-memory caches. Both support TTL and key-based lookups.

Cache layers

Caching happens at many layers: browser, CDN, reverse proxy, application, and database. Each layer reduces load on the layers below it.

Figure 6

- Browser caches static assets and API responses based on headers you send.
- CDN caches static files at edge locations near users. Covers images, CSS, JS, and downloads.
- Reverse proxy caches full HTTP responses before they reach your application.
- Application cache stores computed results in memory or Redis.
- Database cache holds query results and prepared statements.
- Each layer has different invalidation rules. Plan how changes propagate through all layers.

Queues

Some work shouldn't happen in a request. Queues let you accept tasks quickly and process them later with workers. Users don't wait for emails, thumbnails, or reports to finish.

Figure 7

The web server enqueues a job and returns immediately. Workers pull jobs from the queue and process them independently. If a worker crashes, the job returns to the queue and another worker picks it up.

- Use queues for email, notifications, thumbnails, reports, and anything slow or unreliable.
- Workers can crash and restart without losing jobs. The queue holds work until someone finishes it.
- Dead letter queues collect jobs that fail repeatedly so you can inspect them later.
- Idempotent workers produce the same result if they run twice. Retries are safe by design.

Content delivery network (CDN)

A CDN caches your static files on servers near users. Instead of every request travelling to your server, nearby edge servers return cached copies. Latency drops and your origin server handles less traffic.

Figure 8

- Users connect to nearby edge servers instead of the origin server.
- Cache static files: images, CSS, JS, fonts, and downloads.
- Set headers to control what gets cached and for how long.
- Invalidate cache when you deploy new versions or need to remove stale data.
- Use a CDN even for your own infrastructure if users are spread across regions.

Considerations of using a CDN

- Cost: bandwidth and requests add up. Cache aggressively to reduce origin traffic.
- Stale content: users may see old files until the cache expires.
- Debugging: headers tell you whether a response came from cache or origin.
- Cache-busting: rename files (usually with hashes) when you deploy updates.
- Not all traffic: dynamic pages and APIs need other caching strategies.

Stateless architecture

Servers shouldn't remember anything between requests. Sessions, uploads, and cache must live outside the web server. Any server can handle any request because none holds unique state.

Figure 9

A request arrives at the load balancer and goes to any web server. The server reads session data from a shared store, stores uploads in object storage, and uses a shared cache. Servers can come and go without users noticing.

- Move sessions to Redis or a database instead of memory.
- Upload files to object storage (S3, GCS) instead of local disk.
- Use shared cache so any server can serve any request.
- Stateless servers make horizontal scaling and deployments much simpler.

Stateless architecture

Figure 10 shows a stateless architecture.

Figure 10

Load balancer, web servers, session store, object storage, and cache. Any server can handle any request because no state lives on the web servers.

Figure 11 shows the stateless architecture.

Figure 11

Load balancer, web servers, session store, object storage, and cache. Any server can handle any request because no state lives on the web servers.

Message queue

Queues let services communicate asynchronously. Instead of calling another service directly, one service drops a message into a queue and moves on. Workers pick up messages when they have capacity. Producers don't wait for consumers.

Figure 12

Recently-used thumbnails stay cached in Redis for quick access. Older thumbnails expire from cache but remain in object storage. Requests for old thumbnails trigger regeneration from the stored original.

- Decouple services so one slow operation doesn't block everything else.
- Workers can retry failed jobs automatically. Combine with idempotent handlers so retries don't duplicate work.
- Queue depth tells you how backlogged the system is. Growing queues mean workers can't keep up.
- Popular options: RabbitMQ, Redis Queue, AWS SQS, Kafka.

Logging, metrics, automation

Logs tell you what happened, metrics tell you how the system is behaving, and automation keeps deployments from being manual and error-prone.

- Structured logs in JSON make searching and filtering easier than plain text.
- Metrics track latency, error rates, and resource usage. Alert on symptoms users feel, not every internal event.
- Health checks let the load balancer remove unhealthy servers automatically.
- Deploy frequently with small changes. Rollbacks are easier when only a few things changed.

Figure 13 shows a stateless architecture.

Figure 13

Load balancer, web servers, session store, object storage, and cache. Any server can handle any request because no state lives on the web servers.

Figure 14 shows a stateless architecture.

Figure 14

Load balancer, web servers, session store, object storage, and cache. Any server can handle any request because no state lives on the web servers.

Database scaling

Read replicas handle most read traffic. Caching removes even more. Eventually the primary database becomes the bottleneck for writes.

Vertical scaling

- Upgrade to a larger machine: more CPU, RAM, and disk.
- Simple and effective for longer than you expect.
- Hardware limits eventually kick in. Even the largest machine can't keep up with enough traffic.

Horizontal scaling

- Add more machines instead of upgrading one machine.
- Requires sharding or distributed storage.
- Adds complexity: cross-shard queries, rebalancing, and consistency across nodes.

Horizontal scaling

Figure 15

Sharding splits the database across multiple machines, each holding a subset of the data. The web server knows which shard holds the data for a given request and queries only that shard.

Figure 16 shows a simple sharding scheme. Each user belongs to one shard based on their user ID. Profiles, posts, and sessions for that user all live on the same shard. Cross-shard queries are rare because most pages only need one user's data.

Figure 16

Figure 17 shows the sharded database.

Figure 17

Pick a sharding key that keeps most queries on one shard. Avoid cross-shard joins and transactions when you can. Rebalancing moves data between shards when one gets too hot or too large.

Hotspots: Celebrity users, trending topics, or viral content can overload one shard while others sit idle. Monitor shard sizes and query rates. Consider splitting hot shards or caching their data aggressively.

Cross-shard queries: Joins across shards are slow and complex. Denormalize data or maintain separate indexes when you need them.

Consistency: Transactions across shards need two-phase commit or eventual consistency. Most systems choose eventual consistency for simplicity and speed.

- Pick a sharding key that matches your most common queries.
- Avoid cross-shard joins and transactions when possible.
- Rebalance when one shard gets too hot or too large.
- Consider managed sharding (Vitess, Citus, DynamoDB) before building your own.

Figure 18

Millions of users and beyond

At this scale, small inefficiencies cost real money. Caching hit rates matter. Database load patterns dictate your architecture. Monitoring tells you what breaks before your users do. Keep deploys small and boring. Complexity compounds faster than traffic.

- Cache aggressively at every layer.
- Shard databases when one machine can't keep up.
- Monitor everything you can measure.
- Keep deploys small and reversible.
- Simplify architecture when complexity outpaces traffic.
- Optimize for the 99th percentile, not just the average.
- Plan for 10x growth but don't build for it yet.

Recommendations and examples will change as your traffic patterns and scale change.

Reference materials

- Scaling Your Rails Application From One Server To Millions (Honeybadger)
- Scaling Up (Mike Perham, Sidekiq)
- Scaling Stack Overflow (Nick Craver, Stack Exchange)
- Engineering Blog (Stack Exchange)
- Scaling Pinterest (Marty Weiner, Pinterest)
- Scaling Instagram (Instagram Engineering)
- Scaling Twitter (Raffi Krikorian, Twitter)
- Scaling Twitter: Making Twitter 10000 Percent Faster (PDF)
- Scaling Memcache at Facebook (PDF)
- TAO: Facebook's Distributed Data Store for the Social Graph (PDF)
- Facebook's Photo Storage (PDF)
- How Dropbox Stores Metadata For Hundreds Of Petabytes (Dropbox)
- How We've Scaled Dropbox (Dropbox)
- Scaling Uber (Uber)
- Scaling Airbnb's Payment Platform (Airbnb)
- Scaling Airbnb's Search (Airbnb)
- Scaling Lessons (Kate Matsudaira)
- Scalability For Dummies (Clay Wilcox)
- Scalability Rules: 50 Principles For Scaling Web Sites (Martin Abbott and Michael Fisher)
- The Art Of Scalability (Martin Abbott and Michael Fisher)
- Web Scalability For Startup Engineers (Artur Ejsmont)
- Designing Data-Intensive Applications (Martin Kleppmann)